

รายงานผลการทดลอง

หาเส้นทางหยิบสินค้าที่สั้นที่สุดในโกดัง

(Warehouse Picking Route Optimization)

สมาชิก

- นายคิตติศิลป์ คงสี รหัสนักศึกษา 6610110425
- นายเจษฎา แก้วมณี รหัสนักศึกษา 6610110049

1. โจทย์ปัญหา

โกดังคือตาราง 2 มิติ $R \times C$: 0 = ทางเดิน, 1 = ชั้นวาง (เดินผ่านไม่ได้) มี entrance 1 จุด, exit_point 1 จุด, pickup_points n จุด (อยู่บนชั้นวาง) พนักงานเริ่มที่ entrance หยิบสินค้าให้ครบทุกจุด (ไม่บังคับลำดับ) แล้วจบที่ exit_point เดิน 4 ทิศทาง ห้ามทแยงมุม ห้ามทะลุชั้นวาง ถือว่าหยิบได้เมื่อยืนบนทางเดินที่ติดกับชั้นวางนั้น

เป้าหมาย: หาเส้นทางระยะรวมน้อยที่สุด และทดลอง optimize ด้วยวิธีอย่างน้อย 2 วิธี

2. หลักการของวิธีที่ใช้ และการประยุกต์กับโจทย์นี้

ทุกวิธีใช้ BFS ทหาระยะทางเดินจริงบน grid และจัดการ เงื่อนไขเฉพาะของโจทย์ 3 ข้อร่วมกัน:

- (1) จุดหยิบ 1 จุดอาจมีทางเดินติดกันได้หลายช่อง → พิจารณาทุกช่องเป็นตัวเลือกจุดยืน
 - (2) ช่องทางเดิน 1 ช่องอาจติดชั้นวางหลายจุด → หยิบได้พร้อมกันหลายจุด (ใช้ bitmask coverage)
 - (3) เป็น Open-path TSP: เริ่ม entrance จบ exit_point ไม่วนกลับจุดเริ่ม
- Cheapest Insertion + 2-opt แทรกจุดหยิบที่เพิ่มระยะน้อยสุดที่ละจุดจนครบ แล้วสลับคู่ขอบด้วย 2-opt เมื่อทำให้สั้นลง ปรับให้เข้ากับโจทย์: 1 การแทรกครอบคลุมได้หลายจุดหยิบพร้อมกันถ้าช่องนั้นติดหลายชั้นวาง
 - Large Neighborhood Search (LNS) ต่อยอดจากผลลัพธ์ข้อบน แล้ววนซ้ำ 300 รอบ: สุ่มตัดจุดแฉะออก (destroy) → แทรกจุดที่ขาดกลับด้วย cheapest insertion (repair) → ปรับต่อยด้วย remove/insert/move/reverse รับเฉพาะเส้นทางที่สั้นลงและหยิบครบ ทำ 2 seed (20, 21) เลือกผลดีที่สุด เป็นการปรับปรุงคุณภาพคำตอบของวิธีข้างต้น
 - Particle Swarm Optimization Topic 10, อนุภาค $2n$ มิติ (ครั้งแรก = ลำดับหยิบแบบ random key, ครั้งหลัง = เลือกช่องยืนหยิบ) 40 particles, 150 รอบ, $w=0.7$, $c1=c2=1.4$ ปรับให้เข้ากับโจทย์: ถ้าช่องที่ถอยตรัสครอบคลุมจุดที่หยิบไปแล้ว จะข้ามจุดนั้น
 - Coverage per Step (วิธีเสริม) greedy เลือกช่องที่ให้ ระยะเดิน ÷ จุดหยิบใหม่ ต่ำสุด ไม่สร้างตารางระยะล่วงหน้า จึงเร็วที่สุด
 - Lazy Insertion + 2-opt (วิธีเสริม) ผลลัพธ์เดียวกับ Cheapest Insertion ทุกประการ แต่คำนวณ BFS เฉพาะช่องที่ใช้จริงแทนตารางระยะครบทุกคู่ → เร็วกว่า เป็นการปรับปรุงความเร็ว ไม่ใช่อัลกอริทึมใหม่

3. ผลลัพธ์ของแต่ละวิธี

ทดสอบด้วยจุดหยิบชุดเดียวกัน (seed = 20) ชุด 10 จุดแรกเป็นส่วนหนึ่งของชุด 20/30/50 จุด

ตาราง: จำนวนก้าว / เวลารวม (วินาที)

จุดหยิบ	Coverage/Step	Lazy Insertion+2opt	Insertion+2opt	PSO	LNS
10	84 / 0.02	76 / 0.02	76 / 0.03	88 / 0.23	72 / 1.36
20	104 / 0.03	96 / 0.05	96 / 0.06	172 / 0.41	76 / 3.53
30	122 / 0.06	118 / 0.07	118 / 0.13	194 / 0.54	88 / 10.06
50	126 / 0.11	108 / 0.23	108 / 0.29	386 / 1.16	94 / 27.48

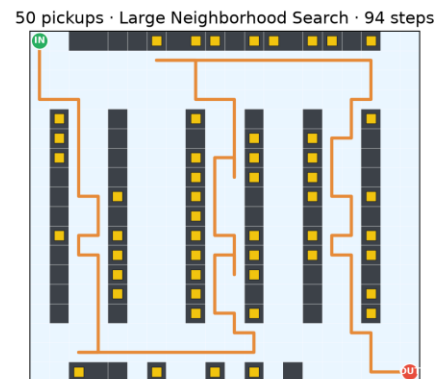
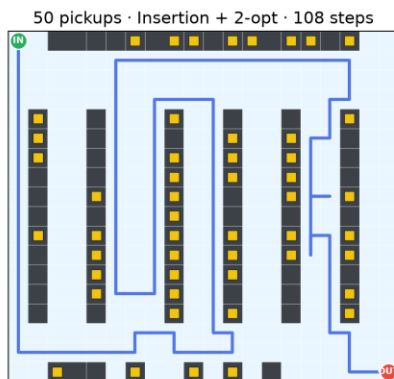
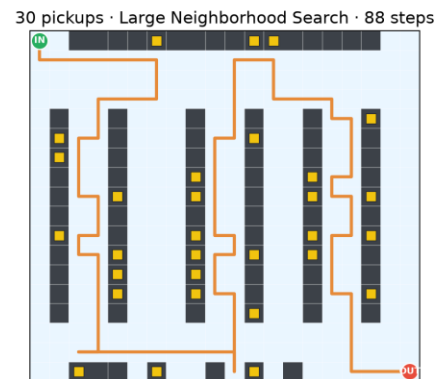
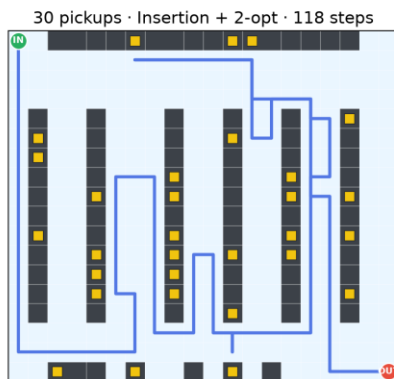
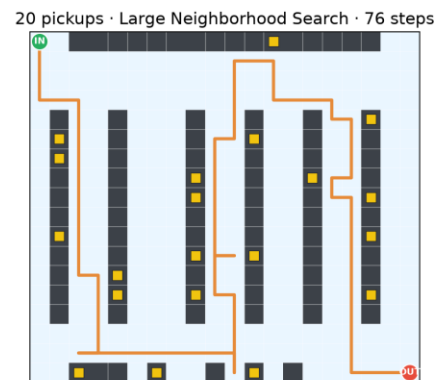
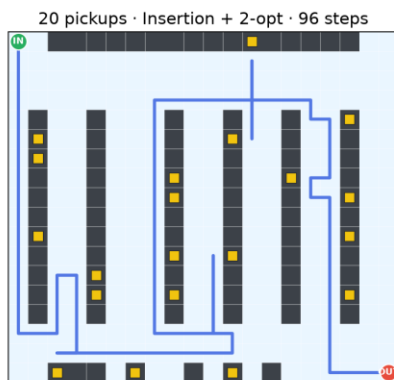
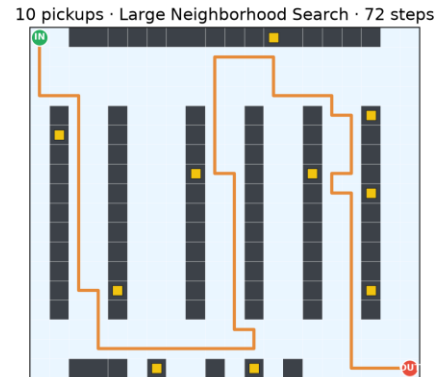
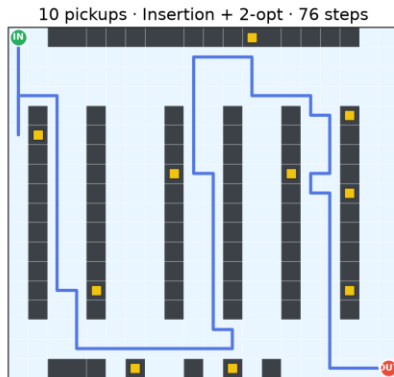
เวลาขึ้นกับสเปกเครื่องที่รัน แนวโน้มเปรียบเทียบระหว่างวิธีคงเดิมในทุกเครื่องที่ทดสอบ

ระยะทางลดลงเมื่อปรับปรุงจาก Insertion+2-opt → LNS

จุดหยิบ	Insertion+2opt	LNS	ลดลง
10	76	72	4
20	96	76	20
30	118	88	30
50	108	94	14

ภาพเส้นทางเปรียบเทียบ

Warehouse routes | yellow: pickups · green: entrance · red: exit



รูปที่ 1: น้ำเงิน = Insertion+2-opt, ส้ม = LNS ที่จุดหยิบ 10/20/30/50 ตามลำดับ

4. เปรียบเทียบผลลัพธ์การทำงาน

เปรียบเทียบ 2 วิธีหลัก: Cheapest Insertion+2opt กับ LNS โดยมี Coverage per Step และ PSO เป็นวิธีเสริม

ประเด็น	Insertion + 2-opt	LNS
ระยะทาง 10 จุด	76 steps	72 steps — ดีกว่า
ระยะทาง 50 จุด	108 steps	94 steps — ดีกว่า ~13%
เวลา 50 จุด	~0.3 s	~27 s — ช้ากว่ามาก
กลไก	แทรกถูกสุด + สลับคู่ขอบ	ทำลาย/ซ่อมแซมซ้ำ + ค้นหาเพื่อนบ้านหลายแบบ
รับประกันคำตอบ	ไม่รับประกัน optimum	ไม่รับประกัน แต่ดีกว่าฐานเสมอ
เหมาะกับ	ต้องการคำตอบเร็ว	คุณภาพสำคัญกว่าความเร็ว

ปัจจัยอื่น: Coverage per Step เร็วสุดทุกขนาดแต่เดินมากกว่า 4–18 ก้าว PSO แ่ลงเร็วเมื่อจุดเพิ่ม

เพราะต้องเลือกทั้งลำดับและจุดยืนพร้อมกัน ทำให้ปริภูมิคำตอบใหญ่เกินจำนวน particle/รอบ ที่ใช้

5. ผลเมื่อเพิ่มจุดหยิบจาก 10 → 20 → 30 → 50

- **Insertion+2-opt:** ระยะทางเพิ่มตามจำนวนจุด (76→96→118→108) เวลาเพิ่มไม่มาก (0.03→0.06→0.13→0.29 s) เพราะความซับซ้อนอยู่ราว $O(n^2-n^3)$ ต่อการแทรก ยังใช้งานได้จริงถึง 50 จุด (ค่าที่ 50 จุดต่ำกว่า 30 จุดเพราะจุดหยิบต่างชุดกัน ไม่ใช่เพราะจุดมากแล้วเส้นทางสั้นลงจริง)
- **LNS:** ดีกว่าฐานทุกขนาด และช่องว่างการปรับปรุงกว้างขึ้นตามจุด (ลด 4/20/30/14 steps) เพราะฐานยังมีจุดพลตมากขึ้นเมื่อจุดมาก แต่เวลาพุ่งเร็วกว่าระยะทางมาก (1.36→3.53→10.06→27.48 s $\approx$ เพิ่ม 20 เท่า เมื่อจุดเพิ่ม 5 เท่า) เพราะแต่ละรอบทำลาย/ซ่อมแซมต้องประเมินตัวเลือกมากขึ้นตามจำนวนจุดยื่นหยิบทั้งหมด
- **สรุป:** ทั้งสองวิธียังใช้งานได้จริงทุกขนาด แต่ trade-off ต่างกัน ต้องการเร็ว/จุดมาก ใช้ Insertion+2-opt หรือ Coverage per Step, ต้องการสั้นที่สุด ใช้ LNS

6. สรุปผลการทดลอง

มองโจทย์เป็น Open-path TSP บนกราฟทางเดินของโกดัง ใช้ BFS หาระยะทางจริง และจัดการเงื่อนไขจุดยื่นหยิบได้หลายช่อง/ช่องเดียวหยิบได้หลายจุดด้วย bitmask coverage

ทดลอง 5 วิธี: Insertion+2opt, PSO, Coverage per Step, LNS, Lazy Insertion

LNS ให้ระยะทางสั้นที่สุดทุกขนาด (72/76/88/94 steps) แลกเวลาที่มากขึ้นชัดเจน

Insertion+2opt สมดุลสุดระหว่างคุณภาพ-ความเร็ว

ส่วน PSO ไม่เหมาะกับโจทย์นี้เมื่อจุดมาก

7. ภาคผนวก

ไฟล์ที่ส่ง (Code)

- `Creat_warehouse.py` — grid, entrance, exit_point, สุ่มจุดหยิบสินค้า
- `optimize_routes.py` — โค้ดหลัก 5 วิธี
- `test_optimize_routes.py` — ชุดทดสอบอัตโนมัติ